

Polymorphic Asynchronous Volatile Field Protocol Fuzzer :

=====

Documentation :

.

Dynamic Network-Field Manipulation System

Overview

A comprehensive Python-based network packet generation system that implements dynamic field manipulation - the ability to have multiple packet fields independently or jointly vary on every evaluation. This system creates real network packets with configurable randomization levels, supports both Layer 2 and Layer 3 transmission, and includes response verification capabilities.

Table of Contents

1. [Core Concepts] (#core-concepts)
2. [Features] (#features)
3. [Installation & Requirements] (#installation--requirements)
4. [Random Generators] (#random-generators)
5. [Packet Classes] (#packet-classes)
6. [Usage Guide] (#usage-guide)
7. [Examples] (#examples)
8. [Technical Architecture] (#technical-architecture)
9. [API Reference] (#api-reference)
10. [Troubleshooting] (#troubleshooting)
11. [Security Considerations] (#security-considerations)

Core Concepts

Volatile Fields

Volatile fields are values that change on every evaluation. The system uses ****VolatileValue**** as the base class for all dynamic values. Each time a field is accessed, it produces a new random value.

Independent Variation

The key capability is not any single random generator, but the ability to have multiple packet fields independently vary on every evaluation. This allows for:

- Changing source/destination addresses
- Varying protocol identifiers
- Randomizing sequence numbers
- Modifying payload lengths and content
- Altering option values
- Changing application identifiers

Fixed vs Randomized Mode

The system supports two operational modes:

- Randomized Mode : All fields change on every packet

- Fixed Mode : Fields remain constant (useful for ICMP ping testing)

Features

Core Features

- ☒ 13 different random generator types
- ☒ Independent field variation
- ☒ Layer 2 (Ethernet) and Layer 3 (IP-only) support
- ☒ Real packet crafting with proper checksums
- ☒ ICMP response listener for delivery verification
- ☒ Configurable destination IP and port
- ☒ Real IP mode for receiving responses
- ☒ Root privilege detection and handling

Random Generators Supported

Generator	Description
-----	-----
RandNum	Random integer in range
RandByte	Random byte (0-255)
RandShort	Random unsigned short
RandInt	Random unsigned int
RandLong	Random unsigned long
RandIP	Random IP address
RandIPv6	Random IPv6 address
RandMAC	Random MAC address
RandUUID	Random UUID
RandString	Random string
RandBin	Random binary data
RandChoice	Random choice from list
RandEnumKeys	Random key from dictionary

Installation & Requirements

Requirements

- Python 3.6+
- Linux operating system (for raw socket support)
- Root/sudo privileges (for packet sending)
- Python standard library modules:
 - `random`, `time`, `struct`, `uuid`, `string`
 - `socket`, `os`, `subprocess`, `re`
 - `select`, `threading`, `typing`

Installation

```
```bash
```

```
Clone or download the script
```

```
chmod +x network_packet_gen.py
```

```
Run with root privileges
```

```
sudo python3 network_packet_gen.py
```

## Dependencies

The script uses only Python standard library - no external dependencies!

---

## Random Generators

### Numeric Generators

#### RandNum

Random integer within a specified range.

```
RandNum(1, 100) # Returns random int between 1 and 100
```

#### RandByte

Random byte value (0-255).

```
RandByte() # Returns random byte
```

#### RandShort

Random unsigned short (0-65535).

```
RandShort() # Returns random short
```

#### RandInt

Random unsigned int (0-4294967295).

```
RandInt() # Returns random int
```

#### RandLong

Random unsigned long (0-18446744073709551615).

```
RandLong() # Returns random long
```

### IP & MAC Generators

#### RandIP

Random IP address from a network range.

```
RandIP("192.168.0.0/16") # Random IP in 192.168.0.0/16
RandIP("10.0.0.0/8") # Random IP in 10.0.0.0/8
```

RandIP6

Random IPv6 address.

```
RandIP6() # Returns random IPv6 like "a1b2:c3d4:e5f6:..."
```

RandMAC

Random MAC address.

```
RandMAC() # Returns random MAC like "aa:bb:cc:dd:ee:ff"
```

String & Binary Generators

RandString

Random string of specified length.

```
RandString(32) # 32-character random string
RandString() # Random length (1-100 characters)
```

RandBin

Random binary data.

```
RandBin(64) # 64 bytes of random binary
RandBin() # Random length
```

Selection Generators

RandChoice

Random choice from a list.

```
RandChoice(6, 17, 1) # Randomly returns 6, 17, or 1
RandChoice('TCP', 'UDP', 'ICMP')
```

RandEnumKeys

Random key from a dictionary.

```
RandEnumKeys({'a': 1, 'b': 2, 'c': 3}) # Randomly returns 'a', 'b', or 'c'
```

## RandUUID

Random UUID generation.

```
RandUUID() # Random UUID v4
RandUUID(version=1) # UUID v1
```

---

## Packet Classes

### VolatilePacket

Creates packets where ALL fields change on every evaluation.

Key Fields:

```
class VolatilePacket:
 # Layer 2 - Ethernet
 self.src_mac = RandMAC() # Random source MAC

 # Layer 3 - IP
 self.src_ip = RandIP("192.168.0.0/16") # Random private IP
 self.protocol = RandChoice(6, 17, 1) # Random protocol
 self.ttl = RandNum(64, 128) # Random TTL
 self.tos = RandNum(0, 255) # Random ToS

 # Layer 4 - Transport
 self.src_port = RandShort() # Random source port

 # TCP specific
 self.seq_num = RandInt() # Random sequence
 self.ack_num = RandInt() # Random ACK
 self.tcp_flags = RandChoice(0x02, 0x10, 0x18) # Random flags

 # Payload
 self.payload_size = RandNum(10, 500) # Random size
 self.payload = RandString() # Random content

 # Identifiers
 self.identifier = RandInt() # Random ID
 self.session_id = RandUUID() # Random UUID
```

### FixedPacket

Creates packets with FIXED (non-randomized) fields - ideal for ICMP ping testing.

Key Features:

- Uses REAL IP address
- Fixed TTL (64)
- ICMP Echo Request only
- Consistent payload
- Fixed identifiers

## ICMPListener

Background listener that captures ICMP echo replies for delivery verification.

Methods:

- `start(interface)` - Start listening
  - `get_responses(timeout)` - Get received responses
  - `stop()` - Stop listening
- 

## Usage Guide

### Interactive Mode

When you run the script, it guides you through configuration:

```
sudo python3 network_packet_gen.py
```

### Step 1: Layer 2 Mode Selection

```
Use Layer 2 (Ethernet) mode? (y/N):
```

- YES (y): Adds Ethernet header with MAC addresses - for local networks
- NO (N): IP-only packets - for external/WAN traffic

### Step 2: Destination Configuration

```
Enter destination IP:
Enter destination port (0 for ICMP):
Number of packets to send (default: 3):
```

### Step 3: Response Reception Mode

```
Use REAL IP to receive responses? (y/N):
```

- YES (y): Uses your real IP, receives responses
- NO (N): Uses random private IP, no responses

## Step 4: Randomization Mode

Use RANDOMIZED fields? (y/N):

- YES (y): All fields randomize on each packet
- NO (N): Fixed fields (ICMP ping)

## Command Line Examples

```
Basic ping test with real IP
sudo python3 network_packet_gen.py
Select: NO (Layer 3), 8.8.8.8, 0, YES (real IP), NO (fixed fields)

Fuzzing with randomized packets
sudo python3 network_packet_gen.py
Select: NO (Layer 3), 192.168.1.1, 80, YES (real IP), YES (randomized)

Local network testing with Layer 2
sudo python3 network_packet_gen.py
Select: YES (Layer 2), 192.168.1.100, 443, YES (real IP), YES (randomized)
```

---

## Examples

### Example 1: Basic ICMP Ping Test

Test if packets reach a destination.

Scenario: Verify connectivity to 8.8.8.8

Configuration:

- Layer 3 (IP-only)
- Destination: 8.8.8.8
- Port: 0 (ICMP)
- Real IP: YES
- Randomized: NO (fixed fields)

Output:

```
🔥 Sending 3 FIXED (non-randomized) packets:
Target: 8.8.8.8
Mode: Layer 3 (IP-only)
Source IP: 192.168.1.100 (REAL IP)
Fields are FIXED - no randomization
```

---

Packet #1:

IP:	192.168.1.100 -> 8.8.8.8
Proto:	ICMP (Echo Request)
TTL:	64
ID:	4660

```
Payload: 32 bytes
✓ Sent
```

```
⌚ Waiting 3.0s for responses...
```

```
🔴 RESPONSES RECEIVED:
```

```
✓ Response received:
 From: 8.8.8.8
 To: 192.168.1.100
 ICMP ID: 4660
 ICMP Seq: 0
 Size: 76 bytes
✓ PACKET REACHED DESTINATION!
```

```
✓ Successfully received 3 responses!
Packets successfully reached the destination!
```

## Example 2: Network Fuzzing with Randomized Packets

Send packets with completely random fields.

Scenario: Fuzz a local web server

Configuration:

- Layer 2 (Ethernet)
- Destination: 192.168.1.10
- Port: 80
- Real IP: YES
- Randomized: YES

Output:

```
🔴 Sending 3 RANDOMIZED packets:
 Target: 192.168.1.10
 Mode: Layer 2 (Ethernet)
 Source IP: REAL IP (will get responses)
 Fields change on EVERY packet!
```

```

Packet #1:
 MAC: 52:54:00:12:34:56 -> aa:bb:cc:dd:ee:ff
 IP: 192.168.1.100:43218 -> 192.168.1.10:80
 Proto: TCP (6)
 TCP: Seq=3423421345, Ack=1234567890, Flags=0x18
 TTL: 72
 ID: 3984273912
 Payload: 234 bytes
 App: HTTP v2.0
✓ Sent
```

```
Packet #2:
 MAC: 52:54:00:12:34:56 -> aa:bb:cc:dd:ee:ff
```



IP:	192.168.1.100:56321 -> 192.168.1.10:80
Proto:	UDP (17)
Ports:	56321 -> 80
TTL:	95
ID:	1245789634
Payload:	412 bytes
App:	DNS v1.0
	Sent

### Example 3: Stealth Mode (Random Private IP)

Send packets with random source IPs (responses go to random IPs).

#### Configuration:

- Layer 3 (IP-only)
- Destination: 8.8.8.8
- Port: 0 (ICMP)
- Real IP: NO
- Randomized: YES

#### Output:

 Sending 3 RANDOMIZED packets:  
Target: 8.8.8.8  
Mode: Layer 3 (IP-only)  
Source IP: RANDOM IP (no responses)  
Fields change on EVERY packet!

---

#### Packet #1:

IP:	192.168.45.23 -> 8.8.8.8
Proto:	ICMP (1)
ICMP:	Echo Request
TTL:	89
ID:	2847391023
Payload:	189 bytes

#### Packet #2:

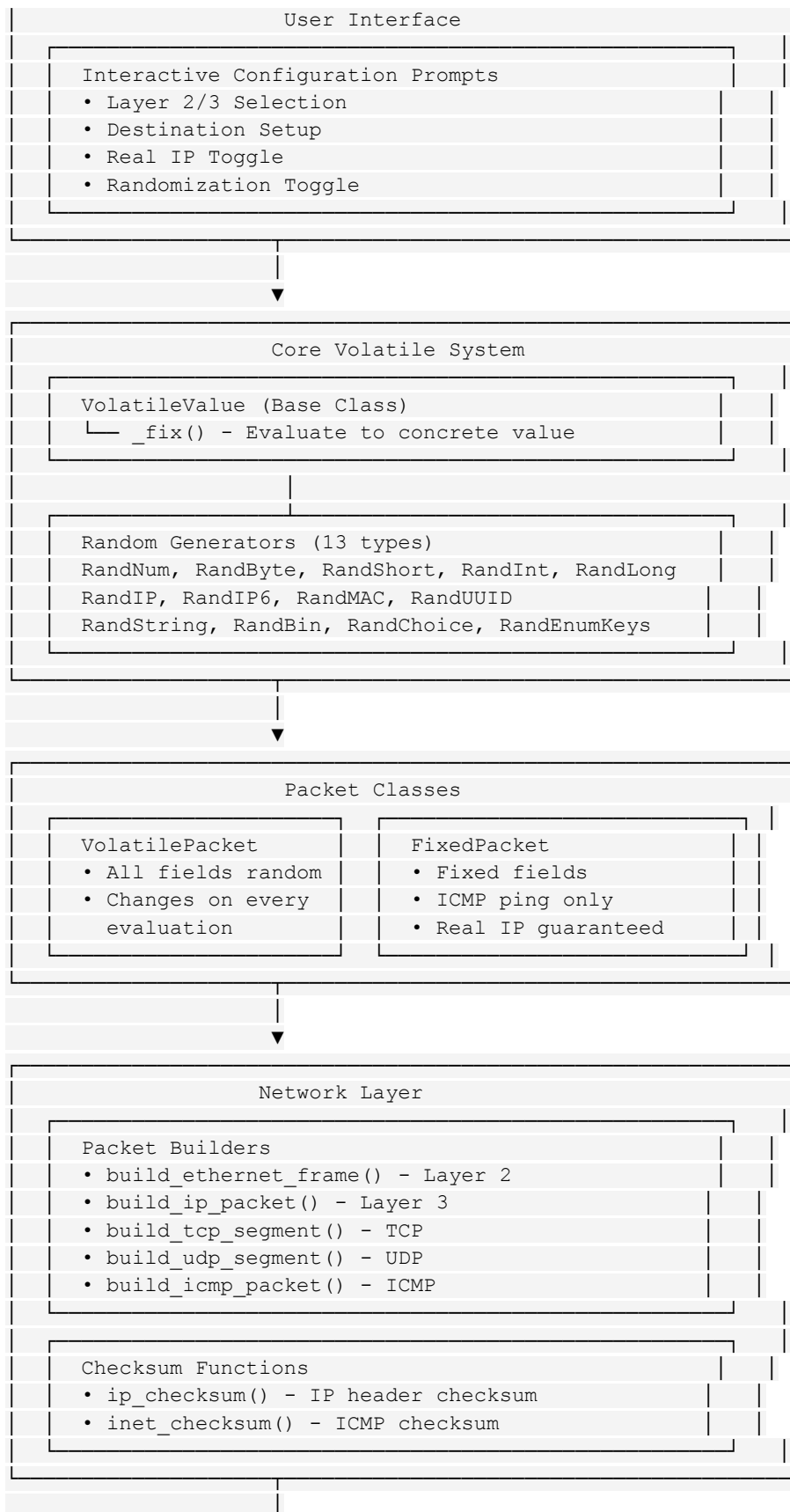
IP:	192.168.178.91 -> 8.8.8.8
Proto:	TCP (6)
Ports:	34567 -> 0
TTL:	117
ID:	3928471029
Payload:	323 bytes

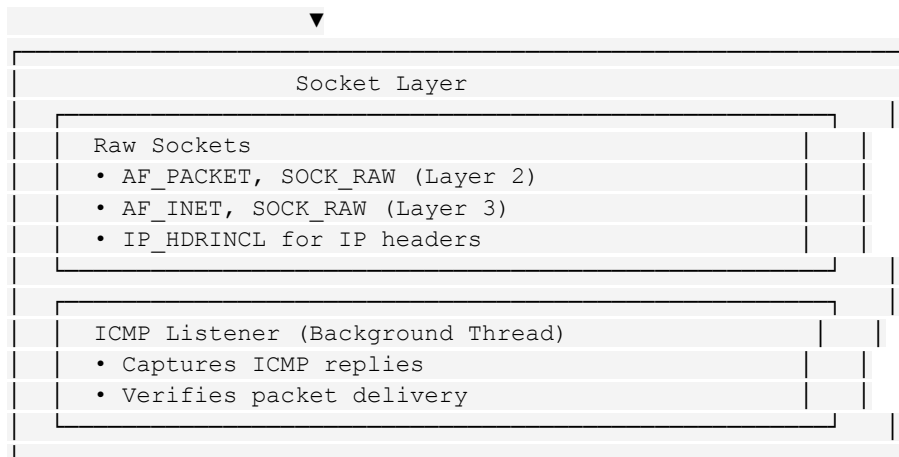
---

## Technical Architecture

### Architecture Diagram

---





## Packet Flow

### 1. Configuration Phase

- User selects Layer 2/3 mode
- Sets destination IP and port
- Chooses real/random IP
- Selects randomization mode

### 2. Packet Generation Phase

- Volatile fields are evaluated
- Random values generated for each field
- Packet is built with proper headers
- Checksums calculated

### 3. Transmission Phase

- Raw socket created
- Packet sent on network
- ICMP listener started (for responses)

### 4. Response Phase

- Listener captures responses
- Delivery verified
- Statistics displayed

---

## API Reference

### Core Classes

VolatileValue (Generic[T])

Base class for all volatile values.

```
class VolatileValue(Generic[T]):
 def _fix(self) -> T: # Evaluate to concrete value
 def __str__(self) -> str: # String representation
 def __bytes__(self) -> bytes: # Bytes representation
```

## Random Generators API

### RandNum

```
RandNum(min_val: int = 0, max_val: int = 100)
Returns: random int in range [min_val, max_val]
```

### RandIP

```
RandIP(network: str = "0.0.0.0/0")
Returns: random IP address in CIDR range
```

### RandMAC

```
RandMAC()
Returns: random MAC address in format "xx:xx:xx:xx:xx:xx"
```

### RandString

```
RandString(size: Optional[int] = None, chars: str = DEFAULT_CHARS)
Returns: random string of specified length
```

### RandChoice

```
RandChoice(*choices: Any)
Returns: random element from choices
```

## Packet Classes API

### VolatilePacket

```
class VolatilePacket:
 def __init__(self, dest_ip: str, dest_port: int,
 use_layer2: bool = True, use_real_ip: bool = False)
 def build_ip_packet(self) -> bytes
 def send_with_response_check(self, count: int = 3, timeout: float = 2.0) -> bool
 def get_current_fields(self) -> Dict[str, Any]
```

## FixedPacket

```
class FixedPacket:
 def __init__(self, dest_ip: str, dest_port: int, use_layer2: bool = True)
 def send_with_response_check(self, count: int = 3, timeout: float = 2.0) -> bool
```

## ICMPListener

```
class ICMPListener:
 def start(self, interface: str = None) -> bool
 def get_responses(self, timeout: float = 2.0) -> List[Dict]
 def stop(self) -> None
```

## Utility Functions

```
def ip_checksum(data: bytes) -> int
 # Calculate IP checksum

def get_local_ip() -> str
 # Get local IP address

def get_public_ip() -> str
 # Get public IP address (via API)

def get_interface_info() -> Dict[str, Any]
 # Get network interface information

def get_dst_mac(ip: str) -> Optional[str]
 # Get MAC address via ARP
```

---

.

=====

## POC :

=====

## Volatility Not used ( response received ) :

→ Downloads `sudo python3 deepseek_python_20260830_5bda6a.py`

=====

 DYNAMIC NETWORK-FIELD MANIPULATION SYSTEM

Independent field randomization on every packet

=====

=====

DYNAMIC NETWORK-FIELD MANIPULATION

All fields vary independently on each evaluation

=====

 Generating 5 packets - ALL fields change on each evaluation:

-----

Field types used:

- RandNum - Random numbers
- RandByte - Random bytes
- RandShort - Random shorts
- RandInt - Random integers
- RandLong - Random longs
- RandIP - Random IP addresses
- RandIP6 - Random IPv6 addresses
- RandMAC - Random MAC addresses
- RandUUID - Random UUIDs
- RandString - Random strings
- RandBin - Random binary data
- RandChoice - Random choices
- RandEnumKeys - Random enum keys

-----

Packet #1:

SRC IP: 192.168.8.109:10679  
DST IP: 8.8.8.8:80 (FIXED)  
PROTO: ICMP (ID: 1)  
TTL: 123  
ID: 4238463213  
SESSION: 0b581668  
APP: HTTP v1.0  
PAYLOAD: 397 bytes

Packet #2:

SRC IP: 192.168.53.199:4665  
DST IP: 8.8.8.8:80 (FIXED)  
PROTO: UDP (ID: 17)  
TTL: 64  
ID: 2392372138  
SESSION: 1fff2950  
APP: SMTP v1.0  
PAYLOAD: 212 bytes

Packet #3:

SRC IP: 192.168.19.92:30116

DST IP: 8.8.8.8:80 (FIXED)  
PROTO: ICMP (ID: 1)  
TTL: 96  
ID: 275625427  
SESSION: baa724d4  
APP: SMTP v1.0  
PAYLOAD: 127 bytes

Packet #4:

SRC IP: 192.168.174.215:41825  
DST IP: 8.8.8.8:80 (FIXED)  
PROTO: TCP (ID: 6)  
TCP: Seq=2587731423, Flags=0x02  
TTL: 110  
ID: 854528597  
SESSION: ba5db777  
APP: HTTPS v3.0  
PAYLOAD: 24 bytes

Packet #5:

SRC IP: 192.168.96.92:9483  
DST IP: 8.8.8.8:80 (FIXED)  
PROTO: ICMP (ID: 1)  
TTL: 122  
ID: 3544240459  
SESSION: 0f5bec6e  
APP: SSH v1.1  
PAYLOAD: 333 bytes

- =====
- ✓ Every evaluation produces DIFFERENT values
  - ✓ All fields are independent and volatile
  - ✓ Destination IP and port remain FIXED (user provided)
- =====

=====

 LAYER 2 (Ethernet) MODE SELECTION

=====

Layer 2 Mode (YES):

- Adds Ethernet header with MAC addresses
- Requires destination MAC (uses ARP)
- Best for: Local network traffic
- Example: 192.168.1.100, 10.0.0.1

Layer 3 Mode (NO):

- Sends IP packets only (no Ethernet header)
- Kernel handles MAC address resolution
- Best for: External/WAN traffic
- Example: google.com, 8.8.8.8, any public IP

-----

Use Layer 2 (Ethernet) mode? (y/N): N

☒ Layer 3 mode selected - IP-only packets  
Suitable for external/WAN traffic

=====

 SET DESTINATION (Fixed fields)  
These fields will NOT change - user provided

=====

Enter destination IP: 3.173.165.22  
Enter destination port (0 for ICMP): 443  
Number of packets to send (default: 3):

=====

 RESPONSE RECEPTION MODE

=====

If you use REAL IP:

- Packets will use your actual IP as source
- Responses will come back to your machine
- You can see if packets reached the destination
- Best for: Testing packet delivery

If you use RANDOM IP:

- Packets will use random private IPs
- Responses will go to the random IP (not your machine)
- You won't see responses
- Best for: Stealth/obfuscation

-----

Use REAL IP to receive responses? (y/N): y

☒ Using REAL IP: 192.168.0.102



Responses will be received and displayed!

=====

 RANDOMIZATION MODE

=====

RANDOMIZED fields (YES):

- ALL fields change on every packet
- Source IP, ports, TTL, payload, etc. all vary
- Each packet is completely different
- Best for: Testing, fuzzing, variety

FIXED fields (NO):

- Fields are fixed/non-random
- Uses REAL IP, fixed TTL, fixed payload
- ICMP ping with consistent values
- Best for: Confirming packet delivery

-----

Use RANDOMIZED fields? (y/N): N

☒ FIXED mode - Fields are constant (ICMP ping)  
This is useful for confirming packet delivery

 Target: 3.173.165.22:443

 Packets: 3

 Mode: Layer 3 (IP-only)

 Source IP: REAL (will get responses)

 Fields: FIXED (ICMP ping)

 Sending 3 FIXED (non-randomized) packets:

Target: 3.173.165.22

Mode: Layer 3 (IP-only)

Source IP: 192.168.0.102 (REAL IP)

Fields are FIXED - no randomization

-----

Packet #1:

IP: 192.168.0.102 -> 3.173.165.22

Proto: ICMP (Echo Request)

TTL: 64

ID: 4660

Payload: 32 bytes

✓ Sent

Packet #2:

IP: 192.168.0.102 -> 3.173.165.22

Proto: ICMP (Echo Request)

TTL: 64

ID: 4660

Payload: 32 bytes

✓ Sent

Packet #3:

IP: 192.168.0.102 -> 3.173.165.22

Proto: ICMP (Echo Request)

TTL: 64

ID: 4660

Payload: 32 bytes

✓ Sent

⌚ Waiting 3.0s for responses...

---

📦 RESPONSES RECEIVED:

---

✓ Response received:

From: 3.173.165.22

To: 192.168.0.102

ICMP ID: 4660

ICMP Seq: 0

Size: 74 bytes

✓ PACKET REACHED DESTINATION!

✓ Response received:

From: 3.173.165.22

To: 192.168.0.102

ICMP ID: 4660

ICMP Seq: 0

Size: 74 bytes

✓ PACKET REACHED DESTINATION!

✓ Response received:

From: 3.173.165.22

To: 192.168.0.102

ICMP ID: 4660

ICMP Seq: 0

Size: 74 bytes

✓ PACKET REACHED DESTINATION!

✓ Successfully received 3 responses!

Packets successfully reached the destination!

=====

✓ COMPLETE

=====

## Volatility used :

→ Downloads sudo python3 deepseek\_python\_20260830\_5bda6a.py

=====

🌐 DYNAMIC NETWORK-FIELD MANIPULATION SYSTEM

Independent field randomization on every packet

=====

DYNAMIC NETWORK-FIELD MANIPULATION

All fields vary independently on each evaluation

=====

📊 Generating 5 packets - ALL fields change on each evaluation:

-----

Field types used:

- RandNum - Random numbers
  - RandByte - Random bytes
  - RandShort - Random shorts
  - RandInt - Random integers
  - RandLong - Random longs
  - RandIP - Random IP addresses
  - RandIP6 - Random IPv6 addresses
  - RandMAC - Random MAC addresses
  - RandUUID - Random UUIDs
  - RandString - Random strings
  - RandBin - Random binary data
  - RandChoice - Random choices
  - RandEnumKeys - Random enum keys
- 

Packet #1:

SRC IP: 192.168.75.214:49557

DST IP: 8.8.8.8:80 (FIXED)  
PROTO: TCP (ID: 6)  
TCP: Seq=886218566, Flags=0x01  
TTL: 118  
ID: 589206184  
SESSION: 8d9a976e  
APP: DNS v4.0  
PAYLOAD: 279 bytes

Packet #2:

SRC IP: 192.168.174.54:23894  
DST IP: 8.8.8.8:80 (FIXED)  
PROTO: TCP (ID: 6)  
TCP: Seq=2074761224, Flags=0x01  
TTL: 126  
ID: 2446421082  
SESSION: 19110b68  
APP: RDP v1.0  
PAYLOAD: 304 bytes

Packet #3:

SRC IP: 192.168.211.71:19794  
DST IP: 8.8.8.8:80 (FIXED)  
PROTO: UDP (ID: 17)  
TTL: 95  
ID: 965074314  
SESSION: e4d965e0  
APP: SMTP v2.0  
PAYLOAD: 380 bytes

Packet #4:

SRC IP: 192.168.35.36:13844  
DST IP: 8.8.8.8:80 (FIXED)  
PROTO: TCP (ID: 6)  
TCP: Seq=3581053868, Flags=0x04  
TTL: 121  
ID: 2327908807  
SESSION: d187d84f  
APP: SMTP v3.0  
PAYLOAD: 273 bytes

Packet #5:

SRC IP: 192.168.99.205:52750  
DST IP: 8.8.8.8:80 (FIXED)

PROTO: ICMP (ID: 1)  
TTL: 76  
ID: 2316043579  
SESSION: b2a6067c  
APP: HTTPS v4.0  
PAYLOAD: 380 bytes

- =====
- ✓ Every evaluation produces DIFFERENT values
  - ✓ All fields are independent and volatile
  - ✓ Destination IP and port remain FIXED (user provided)
- =====

=====

 LAYER 2 (Ethernet) MODE SELECTION

=====

Layer 2 Mode (YES):

- Adds Ethernet header with MAC addresses
- Requires destination MAC (uses ARP)
- Best for: Local network traffic
- Example: 192.168.1.100, 10.0.0.1

Layer 3 Mode (NO):

- Sends IP packets only (no Ethernet header)
- Kernel handles MAC address resolution
- Best for: External/WAN traffic
- Example: google.com, 8.8.8.8, any public IP

-----

Use Layer 2 (Ethernet) mode? (y/N): N

- ☒ Layer 3 mode selected - IP-only packets  
Suitable for external/WAN traffic

=====

 SET DESTINATION (Fixed fields)

These fields will NOT change - user provided

=====

Enter destination IP: 3.173.165.22

Enter destination port (0 for ICMP): 443

Number of packets to send (default: 3):

=====

## RESPONSE RECEPTION MODE

=====

If you use REAL IP:

- Packets will use your actual IP as source
- Responses will come back to your machine
- You can see if packets reached the destination
- Best for: Testing packet delivery

If you use RANDOM IP:

- Packets will use random private IPs
- Responses will go to the random IP (not your machine)
- You won't see responses
- Best for: Stealth/obfuscation

-----

Use REAL IP to receive responses? (y/N): N

☒ Using RANDOM IP (private range)  
Responses will NOT be received

=====

## RANDOMIZATION MODE

=====

RANDOMIZED fields (YES):

- ALL fields change on every packet
- Source IP, ports, TTL, payload, etc. all vary
- Each packet is completely different
- Best for: Testing, fuzzing, variety

FIXED fields (NO):

- Fields are fixed/non-random
- Uses REAL IP, fixed TTL, fixed payload
- ICMP ping with consistent values
- Best for: Confirming packet delivery

-----

Use RANDOMIZED fields? (y/N): y

✅ RANDOMIZED mode - All fields vary on each packet!

🎯 Target: 3.173.165.22:443

📦 Packets: 3

🔧 Mode: Layer 3 (IP-only)

📡 Source IP: RANDOM (no responses)

🎲 Fields: RANDOMIZED

📡 Sending 3 RANDOMIZED packets:

Target: 3.173.165.22

Mode: Layer 3 (IP-only)

Source IP: RANDOM IP (no responses)

Fields change on EVERY packet!

---

Packet #1:

IP: 192.168.182.150 -> 3.173.165.22

Proto: UDP (17)

Ports: 54669 -> 443

TTL: 100

ID: 2304815025

Payload: 80 bytes

App: HTTP v4.0

✅ Sent

Packet #2:

IP: 192.168.28.10 -> 3.173.165.22

Proto: UDP (17)

Ports: 838 -> 443

TTL: 84

ID: 3336837323

Payload: 317 bytes

App: RDP v3.0

✅ Sent

Packet #3:

IP: 192.168.18.121 -> 3.173.165.22

Proto: ICMP (1)

ICMP: Echo Request

TTL: 108

ID: 2947781630

Payload: 476 bytes

App: HTTP v4.0

✅ Sent

⌚ Waiting 3.0s for responses...

📧 RESPONSES RECEIVED:

❌ No responses received!

Possible reasons:

- Target is not reachable
- Firewall is blocking packets
- Target doesn't respond to this protocol
- Using random IP (try 'y' for real IP)

✅ COMPLETE

➔ Downloads

## Traffic analysis

➔ Downloads `sudo tcpdump -i wlp2s0 dst 3.173.165.22`

tcpdump: verbose output suppressed, use -v[v]... for full protocol decode

listening on wlp2s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes

18:35:25.019001 IP larryC6 > server-3-173-165-22.ams56.r.cloudfront.net: ICMP echo request, id 4660, seq 0, length 40

18:35:25.119305 IP larryC6 > server-3-173-165-22.ams56.r.cloudfront.net: ICMP echo request, id 4660, seq 0, length 40

18:35:25.219643 IP larryC6 > server-3-173-165-22.ams56.r.cloudfront.net: ICMP echo request, id 4660, seq 0, length 40

18:37:17.741658 IP 192.168.150.132.53898 >  
server-3-173-165-22.ams56.r.cloudfront.net.https: Flags [.], seq 2935196867:2935196900, ack 3394841019, win 65535, length 33

18:37:17.842450 IP 192.168.159.116.12987 >  
server-3-173-165-22.ams56.r.cloudfront.net.https: UDP, length 403



18:37:17.942941 IP 192.168.60.36.11904 > server-3-173-165-22.ams56.r.cloudfront.net.https:  
UDP, length 94

^C

6 packets captured

6 packets received by filter

0 packets dropped by kernel

=====

```
larry@larryC6:~/Downloads larry@larryC6:~/Downloads x
→ Downloads sudo tcpdump -i wlp2s0 dst 3.173.165.22
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on wlp2s0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:35:25.019001 IP larryC6 > server-3-173-165-22.ams56.r.cloudfront.net: ICMP echo request, id 4660, seq 0, length 40
18:35:25.119305 IP larryC6 > server-3-173-165-22.ams56.r.cloudfront.net: ICMP echo request, id 4660, seq 0, length 40
18:35:25.219643 IP larryC6 > server-3-173-165-22.ams56.r.cloudfront.net: ICMP echo request, id 4660, seq 0, length 40
18:37:17.741658 IP 192.168.150.132.53898 > server-3-173-165-22.ams56.r.cloudfront.net.https: Flags [.], seq 2935196867:2935196900, ack 3394841019, win 65535, length 33
18:37:17.842450 IP 192.168.159.116.12987 > server-3-173-165-22.ams56.r.cloudfront.net.https: UDP, length 403
18:37:17.942941 IP 192.168.60.36.11904 > server-3-173-165-22.ams56.r.cloudfront.net.https: UDP, length 94
^C
6 packets captured
6 packets received by filter
0 packets dropped by kernel
→ Downloads
```

📦 Packets: 3

🔧 Mode: Layer 3 (IP-only)

📡 Source IP: REAL (will get responses)

🌐 Fields: FIXED (ICMP ping)

📦 Sending 3 FIXED (non-randomized) packets:

Target: 3.173.165.22

Mode: Layer 3 (IP-only)

Source IP: 192.168.0.102 (REAL IP)

Fields are FIXED - no randomization

-----

Packet #1:

IP: 192.168.0.102 -> 3.173.165.22

Proto: ICMP (Echo Request)

TTL: 64

ID: 4660

Payload: 32 bytes

✅ Sent

Packet #2:

IP: 192.168.0.102 -> 3.173.165.22

Proto: ICMP (Echo Request)

TTL: 64

ID: 4660

Payload: 32 bytes

✅ Sent

Packet #3:

IP: 192.168.0.102 -> 3.173.165.22

Proto: ICMP (Echo Request)

TTL: 64

ID: 4660

Payload: 32 bytes

✅ Sent

⌚ Waiting 3.0s for responses...

-----  
 RESPONSES RECEIVED:  
-----

✓ Response received:

From: 3.173.165.22  
To: 192.168.0.102  
ICMP ID: 4660  
ICMP Seq: 0  
Size: 74 bytes

✓ PACKET REACHED DESTINATION!

✓ Response received:

From: 3.173.165.22  
To: 192.168.0.102  
ICMP ID: 4660  
ICMP Seq: 0  
Size: 74 bytes

✓ PACKET REACHED DESTINATION!

✓ Response received:

From: 3.173.165.22  
To: 192.168.0.102  
ICMP ID: 4660  
ICMP Seq: 0  
Size: 74 bytes

✓ PACKET REACHED DESTINATION!

✓ Successfully received 3 responses!  
Packets successfully reached the destination!

=====

✓ COMPLETE

=====

→ Downloads sudo python3 deepseek\_python\_20260830\_5bda6a.py

=====

🌐 DYNAMIC NETWORK-FIELD MANIPULATION SYSTEM  
Independent field randomization on every packet

## Packet #1:

IP: 192.168.182.150 -> 3.173.165.22  
Proto: UDP (17)  
Ports: 54669 -> 443  
TTL: 100  
ID: 2304815025  
Payload: 80 bytes  
App: HTTP v4.0  
✓ Sent

## Packet #2:

IP: 192.168.28.10 -> 3.173.165.22  
Proto: UDP (17)  
Ports: 838 -> 443  
TTL: 84  
ID: 3336837323  
Payload: 317 bytes  
App: RDP v3.0  
✓ Sent

## Packet #3:

IP: 192.168.18.121 -> 3.173.165.22  
Proto: ICMP (1)  
ICMP: Echo Request  
TTL: 108  
ID: 2947781630  
Payload: 476 bytes  
App: HTTP v4.0  
✓ Sent

⌚ Waiting 3.0s for responses...

-----  
📡 RESPONSES RECEIVED:  
-----

✗ No responses received!